Upload to Google Colab

```
from google.colab import files

uploaded = files.upload()
```

Select:

```
retail_transactions_100k.csv
```

Then read it:

```
df = spark.read.csv(
    "retail_transactions_100k.csv",
    header=True,
    inferSchema=True
)
```

Display:

```
df.show()
```

Check schema:

```
df.printSchema()
```

Check number of records:

```
df.count()
```

You should get:

```
100000
```

Exercise Set 1 — Basic DataFrame Operations

1. Display the first 20 records.
2. Display the schema.
3. Display all column names.
4. Count the total number of records.
5. Display only:
    - order ID
    - product name
    - city
    - quantity
    - unit price
6. Display only Electronics transactions.
7. Display transactions from Hyderabad.
8. Display transactions where quantity is greater than 3.
9. Display transactions where unit price is greater than ₹20,000.
10. Display Delivered transactions.

Exercise Set 2 — Multiple Conditions

11. Find Electronics transactions from Hyderabad.
12. Find Delivered transactions from the South region.
13. Find orders having quantity greater than 2 and unit price greater than ₹10,000.
14. Find transactions from either Mumbai or Pune.
15. Find transactions that are not Cancelled.
16. Find Furniture products costing more than ₹8,000.
17. Find Accessories products with quantity greater than 3.

Exercise Set 3 — Selecting and Renaming

18. Select only:

```
order_id
customer_name
product_name
unit_price
```

19. Rename:

```
unit_price
```

to:

```
price
```

20. Rename:

```
customer_name
```

to:

```
customer
```

21. Drop the `rating` column.

22. Drop:

```
customer_name
rating
payment_method
```

from a new DataFrame.

Exercise Set 4 — `withColumn()`

Create:

```
gross_amount
```

using:

```
quantity × unit_price
```

23. Add the `gross_amount` column.

24. Create:

```
discount_amount
```

using:

```
gross_amount × discount_pct / 100
```

25. Create:

```
final_amount
```

using:

```
gross_amount - discount_amount
```

26. Add a constant column:

```
country = India
```

27. Add:

```
high_value
```

where:

```
final_amount >= 50000 → Yes
otherwise             → No
```

Exercise Set 5 — Conditional Columns

Create a column:

```
order_size
```

using:

```
final_amount >= 50000 → High
final_amount >= 15000 → Medium
otherwise             → Low
```

28. Create `order_size`.

29. Create another column:

```
quantity_type
```

where:

```
quantity >= 4 → Bulk
otherwise     → Regular
```

30. Create a delivery-status category where:

```
Delivered          → Completed
Cancelled/Returned → Failed
Others             → In Progress
```

Exercise Set 6 — Sorting

31. Sort transactions by unit price from highest to lowest.
32. Sort transactions by quantity from highest to lowest.
33. Sort transactions alphabetically by city.
34. Sort first by region and then by final amount descending.
35. Display the 10 highest-value transactions.

Exercise Set 7 — Distinct Values

36. Display all unique cities.
37. Display all unique regions.
38. Display all unique product categories.
39. Display all unique payment methods.
40. Count how many unique customers exist.
41. Count how many unique products exist.

Typical operations you can introduce here:

```
df.select("city").distinct().show()
```

and:

```
df.select("customer_id").distinct().count()
```

Exercise Set 8 — Aggregations

42. Find total quantity sold.
43. Find total gross sales.
44. Find average unit price.
45. Find maximum unit price.
46. Find minimum unit price.
47. Find total discount given.
48. Find total final sales value.

This lets you introduce:

```
from pyspark.sql.functions import sum, avg, min, max
```

Exercise Set 9 — `groupBy()`

49. Count transactions by status.
50. Count transactions by category.
51. Count transactions by city.
52. Calculate total sales by region.
53. Calculate total sales by category.
54. Calculate average unit price by category.

55. Calculate total quantity sold by product.
56. Calculate total sales by payment method.
57. Find average rating by product.
58. Find total orders by city and status.

Exercise Set 10 — Group and Sort

59. Find total sales by city and sort highest to lowest.
60. Find total quantity sold by product and sort highest to lowest.
61. Find category-wise average unit price and sort descending.
62. Find the top 5 cities based on total sales.
63. Find the top 5 products based on quantity sold.
64. Find the top 10 customers based on purchase amount.

Exercise Set 11 — Dates

First convert:

```
order_date
```

to a proper date if necessary.

Create:

```
year
month
day
```

65. Extract year.
66. Extract month.
67. Extract day.

68. Find the number of transactions by month.
69. Find total sales by month.
70. Find monthly sales by category.
71. Find monthly count by status.
72. Find the month having the highest sales.

Exercise Set 12 — Missing Values

There are intentionally some missing values.

73. Find records where `payment_method` is null/missing.
74. Count missing payment methods.
75. Find records where `rating` is missing.
76. Count missing ratings.
77. Replace missing payment methods with:

```
Unknown
```

78. Replace missing ratings with `0`.
79. Drop records having missing payment methods.